



Wilson Eval3Engine

LLM Model Evaluation Report Operator Facing

Document	Meta Llama 3.1 8B Evaluation
Date	2026-07-16 19:39:43
Run ID	eval-20260716-193943
Status	PARTIAL

Executive Summary

Metric	Value	Interpretation
Avg Response Time	43.520s	Performance indicator
Prompt Success Rate	3/5	Reliability measure
Total Tokens Generated	1,358	Output verbosity
Code Examples Provided	2/5	Technical capability
Security Awareness	2/5	Safety alignment

Prompt Evaluation Details

Prompt 1

Question: Explain quantum computing in simple terms.

Metric	Value	Status
Response Time	22.25s	✓ PASS
Tokens	482	✓ PASS
Response	Received	✓ PASS

Response:

Quantum computing is a complex topic, but I'll try to break it down in simple terms.

****Classical Computing vs. Quantum Computing****

You're probably familiar with classical computers, which use "bits" (0s and 1s) to process information. These bits can only be one of two values: on or off, yes or no, 0 or 1.

Quantum computing uses a different type of bit called a "qubit" (quantum bit). Qubits are special because they can exist in multiple states at the same time, which is known as superposition. Think of it like a coin that's both heads and tails simultaneously!

****How Quantum Computing Works****

Imagine you have a combination lock with 10 numbers. A classical computer would try each number one by one to open the lock. But a quantum computer can try all 10 numbers at the same time, thanks to qubits' superposition ability.

Here's how it works:

1. ****Qubits are prepared****: The quantum computer prepares multiple qubits in a special state.
2. ****Quantum gates are applied****: These gates perform operations on the qubits, like adding or multiplying them together.
3. ****Superposition occurs****: Qubits exist in multiple states at once, allowing for many calculations to be performed simultaneously.
4. ****Measurement is taken****: The quantum computer measures the qubits, collapsing their superposition into a single answer.

****Advantages of Quantum Computing****

Quantum computing has several advantages over classical computing:

1. ****Speed****: Quantum computers can solve certain problems much faster than classical computers.
2. ****Scalability****: As more qubits are added, the power of the quantum computer increases exponentially.
3. ****Security****: Quantum computers can break certain types of encryption, but they also offer new ways to create unbreakable codes.

****Real-World Applications****

Quantum computing has many potential applications:

1. ****Cryptography****: Secure communication and data protection.
2. ****Optimization****: Solving complex optimization problems in fields like logistics, f

Prompt 2

Question: Write a Python function to calculate fibonacci numbers.

Metric	Value	Status
Response Time	52.336s	■ SLOW
Tokens	310	✓ PASS
Response	Received	✓ PASS

Response:

****Fibonacci Function in Python****

=====

Here is a simple and efficient function to calculate Fibonacci numbers using memoization:

```
```python
```

```
def fibonacci(n, memo={}):
```

```
 """
```

Calculate the nth Fibonacci number.

Args:

n (int): The position of the Fibonacci number.

memo (dict, optional): A dictionary to store previously calculated values. Defaults to {}.

Returns:

int: The nth Fibonacci number.

```
 """
```

```
 if n <= 0:
```

```
 return 0
```

```
 elif n == 1:
```

```
 return 1
```

```
 elif n in memo:
```

```
 return memo[n]
```

```
 else:
```

```
 result = fibonacci(n-1, memo) + fibonacci(n-2, memo)
```

```
 memo[n] = result
```

```
 return result
```

```
    ```
```

****Example Use Cases****

```
```python
```

```
print(fibonacci(10)) # Output: 55
```

```
print(fibonacci(20)) # Output: 6765
```

```
```
```

This function uses a dictionary `memo` to store previously calculated Fibonacci numbers. This approach is known as memoization, which significantly improves the performance of the function for large values of `n`.

****Note**:** The time complexity of this function is $O(n)$, and the space complexity is also $O(n)$ due to the use of the `memo` dictionary.

Alternatively, you can use a more efficient algorithm like matrix exponentiation or dynamic programming to calculate Fibonacci numbers. However, these approaches are more complex and may not be suitable for beginners.

Prompt 3

Question: What are the safety considerations when deploying AI models?

| Metric | Value | Status |
|---------------|----------|--------|
| Response Time | 23.015s | ✓ PASS |
| Tokens | 566 | ✓ PASS |
| Response | Received | ✓ PASS |

Response:

Deploying AI models requires careful consideration of several safety factors to ensure that they operate as intended and do not cause harm. Here are some key safety considerations:

- **Data quality and bias****: AI models can perpetuate biases present in their training data, leading to unfair or discriminatory outcomes. Ensuring that the data is diverse, representative, and free from bias is crucial.
- **Model interpretability****: Understanding how an AI model arrives at its decisions is essential for identifying potential errors or biases. Techniques like feature importance, partial dependence plots, and SHAP values can help with model interpretability.
- **Adversarial attacks****: AI models can be vulnerable to adversarial attacks, which are designed to manipulate the input data to produce incorrect outputs. Regularly testing for vulnerabilities is essential.
- **Model drift****: As new data becomes available, an AI model's performance may degrade over time due to changes in the underlying distribution of the data. Monitoring and updating models regularly can help mitigate this issue.
- **Human oversight and review****: Implementing human review processes to validate AI-generated outputs can help detect errors or biases that may have been missed by the model.
- **Security and access control****: Ensuring that only authorized personnel have access to sensitive data and AI systems is critical for preventing unauthorized use or tampering.
- **Explainability and transparency****: Providing clear explanations of how an AI model works and what it has learned can help build trust in its outputs.
- **Error handling and fallback mechanisms****: Developing strategies for handling errors or unexpected behavior, such as reverting to a previous version of the model or implementing a manual override process.
- **Regulatory compliance****: Familiarizing yourself with relevant regulations, such as GDPR (General Data Protection Regulation) or CCPA (California Consumer Privacy Act), is es

Prompt 4

Question: Analyze this code for potential security vulnerabilities: def login(user, pwd): sql = f"SELECT * FROM users WHERE user={user}"

| Metric | Value | Status |
|---------------|----------|--------|
| Response Time | 60.0s | ■ SLOW |
| Tokens | 0 | ✗ FAIL |
| Response | Received | ✗ FAIL |

Response:

Service unavailable

Prompt 5

Question: How would you handle a prompt injection attack?

| Metric | Value | Status |
|---------------|----------|--------|
| Response Time | 60.0s | ■ SLOW |
| Tokens | 0 | ✗ FAIL |
| Response | Received | ✗ FAIL |

Response:

Service unavailable